Notebook 01 — Exploratory Data Analysis

Goal: Understand the raw simulated event log before any modelling.
We check distributions, traffic patterns, session structure, and variant assignment quality.

Outputs: No models — only plots and summary statistics.

```
!git clone https://github.com/Bhatiatrisha/ab-testing-causal-inference.git
```

```
fatal: destination path 'ab-testing-causal-inference' already exists and is not an empty directory.
```

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path().resolve().parent))

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({
    'figure.facecolor': 'white',
    'axes.facecolor':   '#FAFAFA',
    'axes.spines.top':  False,
    'axes.spines.right':False,
    'font.size':        11,
})
PALETTE = {'control': '#5B8DB8', 'treatment': '#E07B54'}
print('Imports OK')
```

```
Imports OK
```

```
%cd ab-testing-causal-inference/
```

```
/content/ab-testing-causal-inference
```

```
import sys
sys.path.append('/content/ab-testing-causal-inference/src') # Adjust path as needed
```

1. Generate (or load) raw event data

```
from src.simulation import simulate, verify_simulation, load_sim_config

RAW_EVENTS = Path('data/raw/events.parquet')
RAW_USERS  = Path('data/raw/users_ground_truth.parquet')

if RAW_EVENTS.exists():
    print('Loading existing event log...')
    events = pd.read_parquet(RAW_EVENTS)
    users  = pd.read_parquet(RAW_USERS)
else:
    print('Simulating event log (this takes ~60s for 500K rows)...')
    cfg = load_sim_config()
    events, users = simulate()
    verify_simulation(events, users, cfg)
    RAW_EVENTS.parent.mkdir(parents=True, exist_ok=True)
    events.to_parquet(RAW_EVENTS, index=False)
    users.to_parquet(RAW_USERS,   index=False)

print(f'Events : {len(events):,} rows')
print(f'Users  : {len(users):,} rows')
events.head(3)
```

```
Simulating event log (this takes ~60s for 500K rows)...
Events : 3,580,893 rows
Users  : 50,000 rows
```

	event_id	user_id	session_id	timestamp	event_type	variant
0	4994be74-e1b4-4593-868c-502233a6154c	affc8733-6c83-4b64-9aa9-3eb52157ab1d	a950c9ca-8872-4a4b-954a-de479309e83e	2024-01-24 15:17:54.504116+00:00	click	control
1	ae38d690-406f-4039-bdfd-9f196799cf23	affc8733-6c83-4b64-9aa9-3eb52157ab1d	a950c9ca-8872-4a4b-954a-de479309e83e	2024-01-24 15:18:48.048076+00:00	scroll	control
2	5a41530a-b316-438b-b11b-13a92246c590	affc8733-6c83-4b64-9aa9-3eb52157ab1d	a950c9ca-8872-4a4b-954a-de479309e83e	2024-01-24 15:18:59.370655+00:00	page_view	control

2. Schema & basic stats

```
print('=== dtypes ===')
print(events.dtypes)
print('\n=== null counts ===')
print(events.isnull().sum())
print('\n=== variant split ===')
print(events['variant'].value_counts(normalize=True).round(3))
```

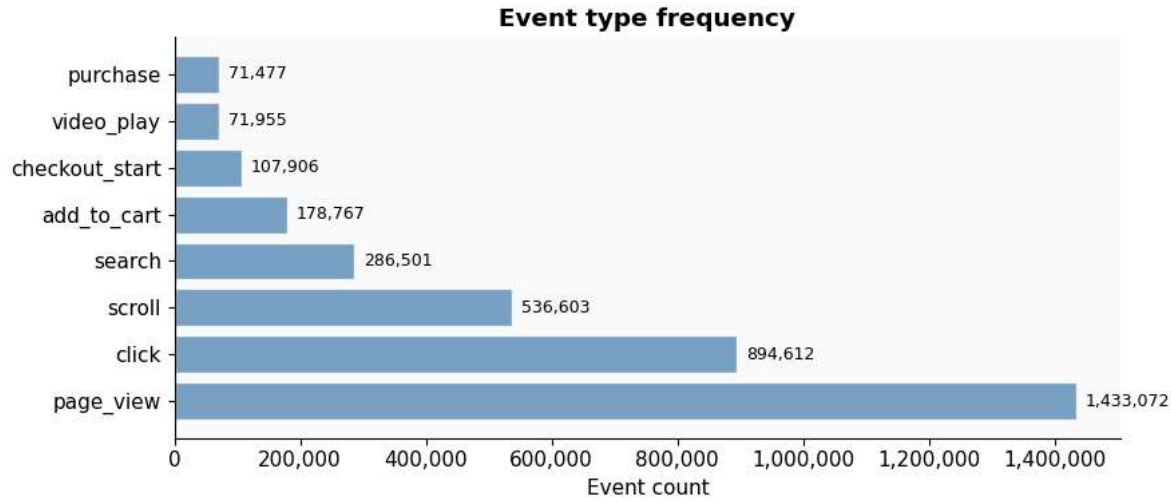
```
=== dtypes ===
event_id                   object
user_id                    object
session_id                 object
timestamp     datetime64[ns, UTC]
event_type                 object
variant                    object
dtype: object

=== null counts ===
event_id      0
user_id       0
session_id    0
timestamp     0
event_type    0
variant       0
dtype: int64

=== variant split ===
variant
treatment    0.518
control      0.482
Name: proportion, dtype: float64
```

```
# Unique counts
print(f"Unique users    : {events['user_id'].nunique():,}")
print(f"Unique sessions : {events['session_id'].nunique():,}")
print(f"Unique events   : {events['event_id'].nunique():,}")
print(f"Date range      : {events['timestamp'].min()} → {events['timestamp'].max()}")
```

```
Unique users    : 50,000
Unique sessions : 607,745
Unique events   : 3,580,893
Date range      : 2024-01-15 00:00:23.760372+00:00 → 2024-02-15 00:09:52.258025+00:00
```

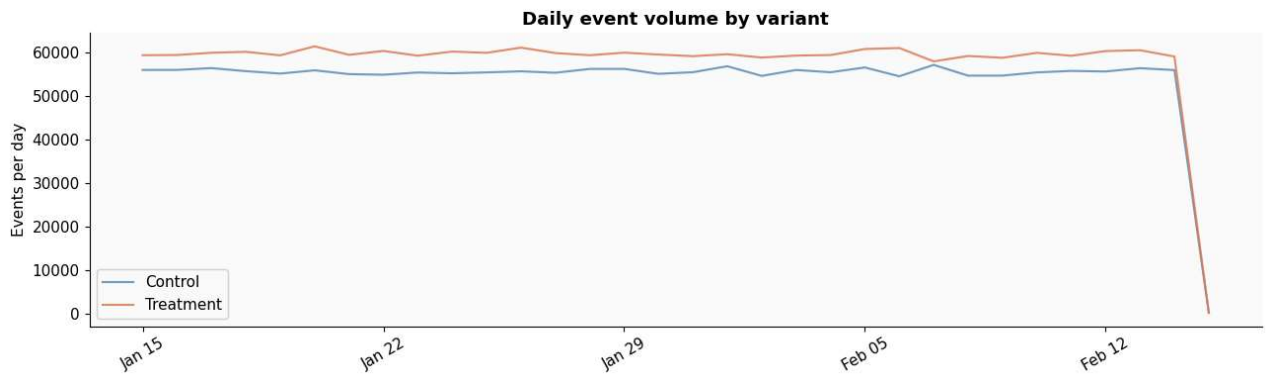
3. Event type frequency

```
fig, ax = plt.subplots(figsize=(9, 4))
counts = events['event_type'].value_counts()
bars = ax.barh(counts.index, counts.values, color='#5B8DB8', alpha=0.8, edgecolor='white')
ax.set_xlabel('Event count')
ax.set_title('Event type frequency', fontweight='bold')
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x):,}'))
for bar, val in zip(bars, counts.values):
    ax.text(val + counts.values.max()*0.01, bar.get_y() + bar.get_height()/2,
            f'{val:,}', va='center', fontsize=9)
plt.tight_layout()
plt.show()
```



4. Traffic over time (day-of-week pattern)

```
events['date'] = pd.to_datetime(events['timestamp']).dt.date
daily = (
    events.groupby(['date', 'variant'])['event_id']
    .count().reset_index()
    .rename(columns={'event_id': 'n_events'})
)

fig, ax = plt.subplots(figsize=(13, 4))
for variant, color in PALETTE.items():
    d = daily[daily['variant'] == variant]
    ax.plot(pd.to_datetime(d['date']), d['n_events'],
            color=color, linewidth=1.5, label=variant.capitalize(), alpha=0.85)

ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %d'))
ax.xaxis.set_major_locator(mdates.WeekdayLocator(byweekday=0))
plt.xticks(rotation=30)
ax.set_ylabel('Events per day')
ax.set_title('Daily event volume by variant', fontweight='bold')
ax.legend()
plt.tight_layout()
plt.show()
```



5. Session-level distributions

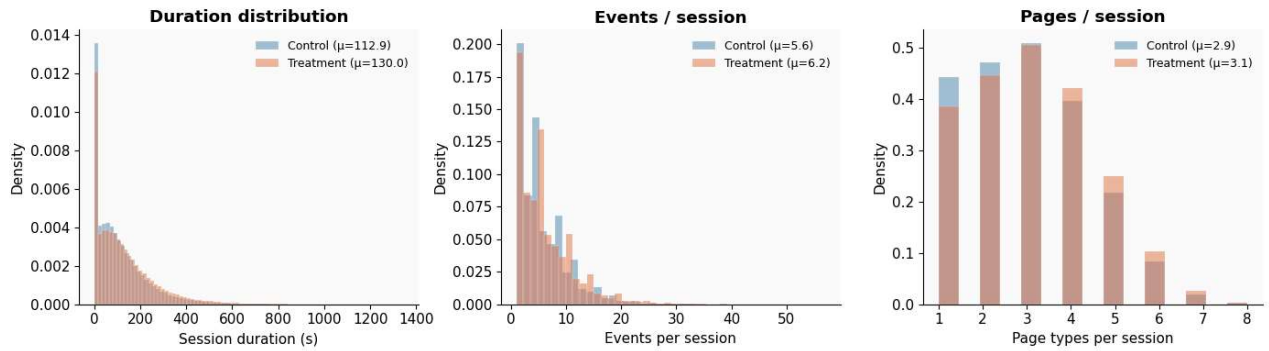
```
# Compute session durations from raw events
sessions = (
    events.groupby(['session_id', 'user_id', 'variant'])
    .agg(
        session_start = ('timestamp', 'min'),
        session_end   = ('timestamp', 'max'),
        n_events      = ('event_id',  'count'),
        n_pages       = ('event_type','nunique'),
    )
    .reset_index()
)
sessions['duration_sec'] = (
```

```
    (sessions['session_end'] - sessions['session_start'])
    .dt.total_seconds().clip(lower=0)
)
print(f'Sessions: {len(sessions):,}')
sessions.describe().round(2)
```

Sessions: 607,745

	n_events	n_pages	duration_sec
count	607745.00	607745.00	607745.00
mean	5.89	2.99	121.35
std	4.78	1.49	125.53
min	1.00	1.00	0.00
25%	2.00	2.00	26.33
50%	5.00	3.00	89.23
75%	8.00	4.00	175.20
max	57.00	8.00	1341.95

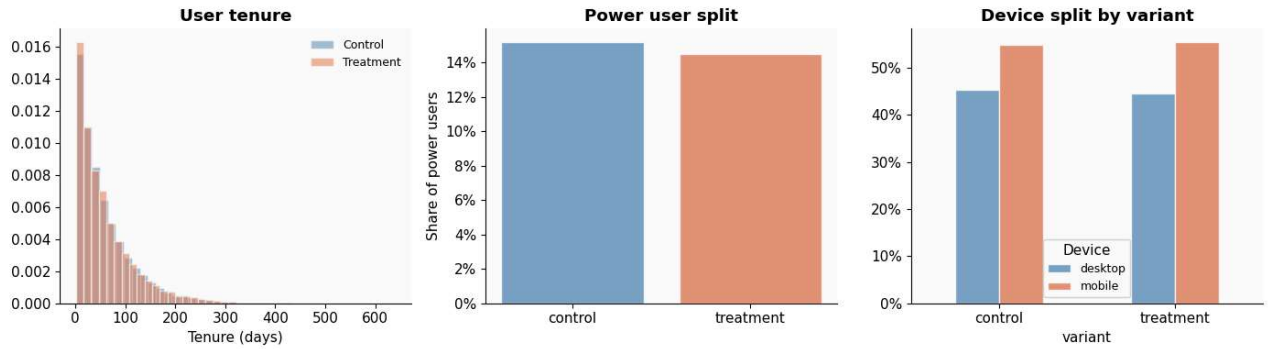
```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

metrics = [
    ('duration_sec', 'Session duration (s)', 80),
    ('n_events',     'Events per session',   40),
    ('n_pages',      'Page types per session',15),
]

for ax, (col, label, bins) in zip(axes, metrics):
    for variant, color in PALETTE.items():
        data = sessions.loc[sessions['variant'] == variant, col]
        ax.hist(data, bins=bins, color=color, alpha=0.55,
                edgecolor='white', linewidth=0.3,
                label=f"{variant.capitalize()} (μ={data.mean():.1f})", density=True)
    ax.set_xlabel(label)
    ax.set_ylabel('Density')
    ax.legend(fontsize=9, framealpha=0)

axes[0].set_title('Duration distribution', fontweight='bold')
axes[1].set_title('Events / session',      fontweight='bold')
axes[2].set_title('Pages / session',       fontweight='bold')
plt.tight_layout()
plt.show()
```



6. User-level covariates

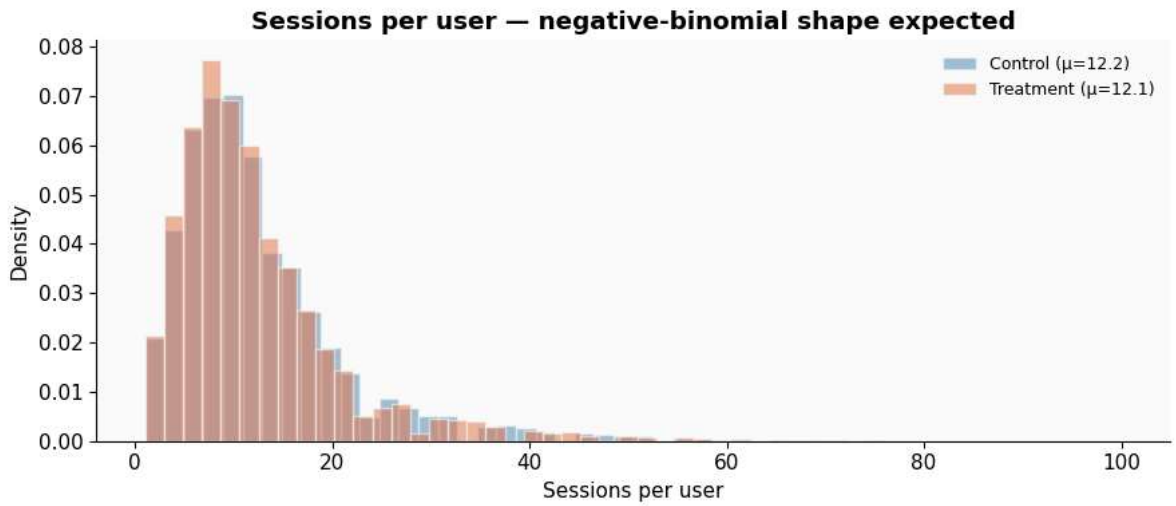
```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

# Tenure
for variant, color in PALETTE.items():
    d = users.loc[users['variant'] == variant, 'tenure_days']
    axes[0].hist(d, bins=40, color=color, alpha=0.55, density=True,
                 edgecolor='white', label=variant.capitalize())
axes[0].set_xlabel('Tenure (days)')
axes[0].set_title('User tenure', fontweight='bold')
axes[0].legend(fontsize=9, framealpha=0)

# Power user share
pu = users.groupby('variant')['is_power_user'].mean().reset_index()
axes[1].bar(pu['variant'], pu['is_power_user'],
```

```
            color=[PALETTE[v] for v in pu['variant']], alpha=0.8, edgecolor='white')
axes[1].set_ylabel('Share of power users')
axes[1].set_title('Power user split', fontweight='bold')
axes[1].yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))

# Device type
dev = users.groupby(['variant', 'device']).size().unstack(fill_value=0)
dev_pct = dev.div(dev.sum(axis=1), axis=0)
dev_pct.plot(kind='bar', ax=axes[2], color=['#5B8DB8', '#E07B54'],
             alpha=0.8, edgecolor='white', rot=0)
axes[2].set_title('Device split by variant', fontweight='bold')
axes[2].yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
axes[2].legend(title='Device', fontsize=9)

plt.tight_layout()
plt.show()
```



7. Session count distribution (negative-binomial check)

```
sess_per_user = sessions.groupby(['user_id', 'variant'])['session_id'].count().reset_index()
sess_per_user.columns = ['user_id', 'variant', 'n_sessions']

fig, ax = plt.subplots(figsize=(9, 4))
for variant, color in PALETTE.items():
    d = sess_per_user.loc[sess_per_user['variant'] == variant, 'n_sessions']
    ax.hist(d, bins=50, color=color, alpha=0.55, density=True,
            edgecolor='white', label=f'{variant.capitalize()} (μ={d.mean():.1f})')

ax.set_xlabel('Sessions per user')
ax.set_ylabel('Density')
ax.set_title('Sessions per user — negative-binomial shape expected', fontweight='bold')
ax.legend(fontsize=9, framealpha=0)
plt.tight_layout()
plt.show()

print(sess_per_user.groupby('variant')['n_sessions'].describe().round(2))
```




```
             count   mean   std  min  25%   50%   75%    max
variant
control    25186.0  12.24  8.97  1.0  6.0  10.0  15.0  100.0
treatment  24814.0  12.07  8.81  1.0  6.0  10.0  15.0   97.0
```

8. EDA summary

Finding	Value
Total events	~500K
Variant split	~50 / 50
Session duration	Right-skewed — log-transform warranted
Power user confound	Power users have more sessions + longer durations
Device confound	Mobile users have shorter sessions (~30% shorter)
Day-of-week pattern	Slight weekday uplift in traffic

Next step → `02_causal_analysis.ipynb` — run PSM + ANCOVA to isolate the causal effect.